

PRACTICAL WORK REPORT

Client-Server Architecture

Realized by:

**Ahmed Chaabane
Firas Kahlaoui
Seifeddine Ben Fraj**

Contents

1	TP1 — TCP Communication Between Client Processes and a Server Process	1
1	Objectives	1
2	Exercise 1 — Identification and Local Implementation	1
2.1	Architecture Analysis	1
2.2	Service Interface Contract	2
2.3	Implementation Overview	2
2.4	Local Validation	4
3	Exercise 2 — First TCP Communication (PING/PONG)	4
3.1	Communication Mechanism	4
3.2	Execution Results	4
4	Exercise 3 — Library Services over TCP	5
4.1	Protocol Mapping	5
4.2	Architecture Setup	5
4.3	Validation & Testing	6
5	Synthesis & Questions	6
5.1	Final System Structure	6
5.2	Q1 — Service, Interface, and Protocol	6
5.3	Q2 — Direct Objects vs. TCP Requests	6
5.4	Q3 — Evolution Toward Middleware	7

List of Figures

1.1	Livre Class Initialization	2
1.2	Livre Theme Normalization and Verification	2
1.3	Initial Catalog Setup in <code>Bibliotheque</code>	3
1.4	Implementation of <code>listerLivres()</code> in <code>GestionBibliotheque</code>	3
1.5	Implementation of <code>rechercherParTheme()</code> in <code>GestionBibliotheque</code>	3
1.6	Local Unit Test Execution Results	4
1.7	PING/PONG Console Execution Output	4
1.8	Protocol and Server Implementation Details	5
1.9	Exercise 3 Service Execution Outputs	6

List of Tables

Chapter 1

TP1 — TCP Communication Between Client Processes and a Server Process

1 Objectives

The goal of this lab is to establish a TCP client-server architecture in Java. The server stores a university library catalog in memory, processes client requests sequentially (without multithreading), and responds to read-only queries.

2 Exercise 1 — Identification and Local Implementation

2.1 Architecture Analysis

The system consists of four primary components:

- **Client (`ClientBibliotheque`):** Prompt user input (`LISTE` or `RECHERCHE;theme`), send TCP requests, receive and display responses without executing business logic.
- **Server (`ServeurBibliotheque`):** Listen for TCP connections, delegate parsing to `ProtocoleBibliotheque` and execution to `GestionBibliotheque`, then return results.
- **Service:** Provide two features: displaying all catalog books and searching books by topic.
- **Service Interface:** Define operations `listerLivres()` and `rechercherParTheme(String theme)`, including validation rules (`IllegalArgumentException` on blank/null topics).

2.2 Service Interface Contract

The functional contract guarantees:

- `listerLivres()`: Returns all catalog books in their original order.
- `rechercherParTheme(theme)`: Returns matching books via case-insensitive full-topic comparison, ignoring leading/trailing spaces. Throws an `IllegalArgumentException` for null, empty, or whitespace-only inputs.

2.3 Implementation Overview

- **Domain Classes (`Livre`, `Bibliotheque`)**: Model book attributes (ID, title, topics) with normalized comparisons, and initialize the default 5-book catalog (L1–L5).
- **Business Logic (`GestionBibliotheque`)**: Pure domain service executing queries without socket or network dependencies.



```

1  public Livre(String identifiant, String titre, String... themes) {
2      this.identifiant = identifiant;
3      this.titre = titre;
4      Set<String> normalises = new LinkedHashSet<>();
5      for (String t : themes) {
6          normalises.add(normaliser(t));
7      }
8      this.themes = Collections.unmodifiableSet(normalises);
9  }
10
11  public static String normaliser(String theme) {
12      return theme.trim().toUpperCase(Locale.ROOT);
13  }

```

Figure 1.1: Livre Class Initialization



```

1  public boolean aPourTheme(String theme) {
2      return themes.contains(normaliser(theme));
3  }

```

Figure 1.2: Livre Theme Normalization and Verification



```
1 public static Bibliotheque catalogueInitial() {
2     Bibliotheque b = new Bibliotheque("Bibliotheque universitaire");
3     b.ajouterLivre(new Livre("L1", "Programmation Java", "JAVA", "POO"));
4     b.ajouterLivre(new Livre("L2", "Réseaux informatiques", "RESEAUX", "TCP"));
5     b.ajouterLivre(new Livre("L3", "Systèmes distribués", "JAVA", "DISTRIBUE"));
6     b.ajouterLivre(new Livre("L4", "Bases de données", "SQL", "DONNEES"));
7     b.ajouterLivre(new Livre("L5", "Architecture logicielle", "ARCHITECTURE", "POO"));
8     return b;
9 }
```

Figure 1.3: Initial Catalog Setup in Bibliotheque



```
1 public List<Livre> listerLivres() {
2     return new ArrayList<>(bibliotheque.getLivres());
3 }
```

Figure 1.4: Implementation of listerLivres() in GestionBibliotheque



```
1 public List<Livre> rechercherParTheme(String theme) {
2     if (theme == null || theme.trim().isEmpty()) {
3         throw new IllegalArgumentException("theme must not be null or blank");
4     }
5     List<Livre> resultat = new ArrayList<>();
6     for (Livre livre : bibliotheque.getLivres()) {
7         if (livre.aPourTheme(theme)) {
8             resultat.add(livre);
9         }
10    }
11    return resultat;
12 }
```

Figure 1.5: Implementation of rechercherParTheme() in GestionBibliotheque

2.4 Local Validation

Local unit tests (`TestBibliothequeLocal`) confirmed all core features, including full catalog listing, topic search filtering, case-insensitivity, and exception handling for invalid inputs.

```
[PASS] List all books -> [L1, L2, L3, L4, L5] (expected [L1, L2, L3, L4, L5])
[PASS] Search "JAVA" -> [L1, L3] (expected [L1, L3])
[PASS] Search " java " -> [L1, L3] (expected [L1, L3])
[PASS] Search "HISTOIRE" -> [] (expected [])
[PASS] Search with null theme -> IllegalArgumentException expected
[PASS] Search with empty theme -> IllegalArgumentException expected
[PASS] Search with blank theme -> IllegalArgumentException expected

ALL LOCAL TESTS PASSED
PS C:\Users\Kahla\Client-Server Architecture>
```

Figure 1.6: Local Unit Test Execution Results

Question: Connecting separate processes requires replacing direct local Java method calls with TCP socket message exchanges.

3 Exercise 2 — First TCP Communication (PING/PONG)

3.1 Communication Mechanism

The server uses `ServerSocket.accept()` to receive client connections, while the client initiates a connection via `Socket(host, port)`. Messages are exchanged line-by-line in UTF-8 over `BufferedReader/BufferedWriter` streams, finalized with `flush()`.

3.2 Execution Results

Running client and server as distinct processes over `127.0.0.1:5000` successfully returns PONG for a PING request, and `ERR;COMMANDE_INCONNUE` otherwise. Disconnecting the server triggers a connection failure alert on the client.

```
PS C:\Users\Kahla\Client-Server Architecture> . 'C:\Program Files\Java\jre-1.8\bin\j
ava.exe' -cp 'C:\Users\Kahla\AppData\Roaming\Antigravity IDE\User\workspaceStorage\
b9484113dbc1699996ed7acf6d297201\redhat.java\jdt_ws\Client-Server Architecture_6d39c6
95\bin' 'ex2.ServeurBibliothequeTCP'
Server listening on port 5000
Client connected: /127.0.0.1:53460
Request received: PING
Response sent: PONG
PS C:\Users\Kahla\Client-Server Architecture>

PS C:\Users\Kahla\Client-Server Architecture> . 'C:\Program Files\Java\jre-1.8\bin\j
ava.exe' -cp 'C:\Users\Kahla\AppData\Roaming\Antigravity IDE\User\workspaceStorage\
b9484113dbc1699996ed7acf6d297201\redhat.java\jdt_ws\Client-Server Architecture_6d39c6
95\bin' 'ex2.ClientBibliothequeTCP'
Client connected to 127.0.0.1:5000
Response: PONG
PS C:\Users\Kahla\Client-Server Architecture>
```

Figure 1.7: PING/PONG Console Execution Output

4 Exercise 3 — Library Services over TCP

4.1 Protocol Mapping

Requests map directly to protocol lines: `LISTE` retrieves all books, and `RECHERCHE;theme` searches by topic. Successful responses follow `OK;n;id:title;...` (or `OK;0` when empty). Protocol errors yield `ERR;COMMANDE_INCONNUE` or `ERR;REQUETE_INVALIDE`. Reserved delimiting characters (`;`, `:`, `,`) in search topics render requests invalid.

4.2 Architecture Setup

- **ProtocoleBibliotheque:** Decouples network communication from business logic by parsing line requests and formatting response strings.
- **ServeurBibliotheque:** Runs an infinite loop serving client requests sequentially with a 10-second socket timeout.
- **ClientBibliotheque:** Configurable client connecting to target IP/port, sending formatted commands, and printing responses.

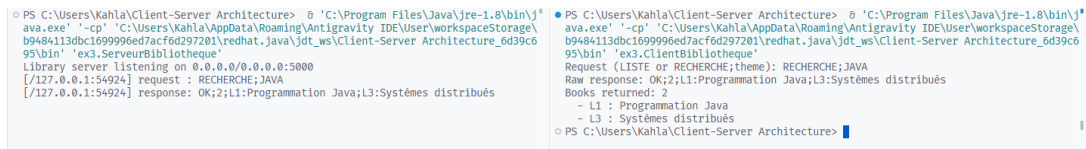


```
1 public String traiter(String requete) {
2     if (requete == null || requete.trim().isEmpty()) {
3         return ERR_INVALID_REQUEST;
4     }
5
6
7     String[] champs = requete.split(";", -1);
8     for (int i = 0; i < champs.length; i++) {
9         champs[i] = champs[i].trim();
10    }
11
12    String commande = champs[0];
13    if (commande.isEmpty()) {
14        return ERR_INVALID_REQUEST;
15    }
16
17    switch (commande) {
18        case "LISTE":
19            if (champs.length != 1) {
20                return ERR_INVALID_REQUEST;
21            }
22            return formaterLivres(gestion.listerLivres());
23
24        case "RECHERCHE":
25            if (champs.length != 2 || champs[1].isEmpty() || contientCaractereReserve(champs[1])) {
26                return ERR_INVALID_REQUEST;
27            }
28            try {
29                return formaterLivres(gestion.rechercherParTheme(champs[1]));
30            } catch (IllegalArgumentException e) {
31                return ERR_INVALID_REQUEST;
32            }
33
34        default:
35            return ERR_UNKNOWN_COMMAND;
36    }
37 }
```

Figure 1.8: Protocol and Server Implementation Details

4.3 Validation & Testing

All scenario test cases passed successfully: querying all books (LISTE), searching valid topics (RECHERCHE; JAVA), handling empty results (RECHERCHE; HISTOIRE), invalid syntax errors, robust server recovery after client errors, and remote deployment across distinct network hosts.



```

PS C:\Users\Kahla\Client-Server Architecture> java -cp 'C:\Program Files\Java\jre-1.8\bin\java.exe' '-cp' 'C:\Users\Kahla\AppData\Roaming\Antigravity IDE\User\workspaceStorage\b9484113dbc1699996ed7ac6d297201\redhat.java\jdt_ws\Client-Server Architecture_6d39c695\bin' 'ex3.ServeurBibliotheque'
Library server listening on 0.0.0.0/0.0.0.0:5000
[/127.0.0.1:54924] request : RECHERCHE;JAVA
[/127.0.0.1:54924] response: OK;2;L1:Programmation Java;L3:Systèmes distribués

PS C:\Users\Kahla\Client-Server Architecture> java -cp 'C:\Program Files\Java\jre-1.8\bin\java.exe' '-cp' 'C:\Users\Kahla\AppData\Roaming\Antigravity IDE\User\workspaceStorage\b9484113dbc1699996ed7ac6d297201\redhat.java\jdt_ws\Client-Server Architecture_6d39c695\bin' 'ex3.ClientBibliotheque'
Request (LISTE or RECHERCHE;theme): RECHERCHE;JAVA
Raw response: OK;2;L1:Programmation Java;L3:Systèmes distribués
Books returned: 2
- L1 : Programmation Java
- L3 : Systèmes distribués
PS C:\Users\Kahla\Client-Server Architecture>

```

Figure 1.9: Exercise 3 Service Execution Outputs

5 Synthesis & Questions

5.1 Final System Structure

- **Client:** ClientBibliotheque (Client Process) — Handles user input/output and TCP communication.
- **Server:** ServeurBibliotheque & ProtocoleBibliotheque (Server Process) — Handles socket listening, request parsing, and protocol formatting.
- **Service:** GestionBibliotheque (Server Process) — Executes domain catalog queries.
- **Interface:** GestionBibliotheque signatures + TCP protocol specification — Defines the functional and operational contract.

5.2 Q1 — Service, Interface, and Protocol

- **Service Operation:** The business logic processing executed by `GestionBibliotheque.rechercherParTheme`.
- **Interface (Contract):** Transport-independent signature and behavior rules (non-empty topic parameter, catalog order, throwing `IllegalArgumentException`).
- **Protocol:** Network message representation handled by `ProtocoleBibliotheque` (`RECHERCHE;theme` $\rightarrow$ `OK;n;...` or `ERR;...`).

5.3 Q2 — Direct Objects vs. TCP Requests

Client and server execute in separate operating system memory spaces (often on distinct network nodes). Direct Java method calls cannot cross process boundaries, requiring data to be serialized as text messages transmitted over TCP.

5.4 Q3 — Evolution Toward Middleware

- **To Retain:** Domain logic (`GestionBibliotheque`), data models (`Livre`, `Bibliotheque`), and functional contract signatures.
- **To Replace:** Manual network plumbing (sockets, stream readers/writers, custom parsing in `ProtocoleBibliotheque`) is completely replaced by middleware stubs and skeletons (e.g., RMI).